

A
Project Proposal
On
**“FINAL YEAR PROJECT DESIGN AND DEVELOPMENT OF AN
E-COMMERCE WEB APPLICATION”**

Submitted to
“METROPOLITAN UNIVERSITY, KARACHI”

Karachi

In partial fulfillment for
Bachelors in Computer Science (BSCS)

Submitted by
Mr. M. WASEEM AKHTAR & Mr. M. ALI DANISH



Under The Guidance of
Prof. SYED IFTEKHARUDDIN

Department of Computer Science
Metropolitan University, Karachi

Session: **2022 - 2025**

INDEX

S. No.	Chapter	Page No
01.	SRS DOCUMENT	
02.	PROJECT DESCRIPTION	
03.	PROJECT WORK LOAD	
04.	PROJECT MEMBERS	
05.	USE CASE DIAGRAM	
06.	ACTIVITY DIAGRAM	
07.	SEQUENCE DIAGRAM	
08.	CLASS DIAGRAM	
09.	ER DIAGRAM	
10.	PROCESS MODEL, SCRUM,/AGILE	
11.	GANTT CHART	

1. SOFTWARE REQUIREMENTS SPECIFICATION (SRS):

An **E-Commerce Software Requirements Specification (SRS)** document serves as the architectural and functional blueprint for building an online store. It aligns developers, designers, and business stakeholders on how the platform will behave, perform, and secure user data. [1, 2, 3, 4, 5]

1. Introduction

1.1 Purpose

This document details the functional and non-functional requirements for the **E-Commerce Platform**. It provides a definitive guide for the development, testing, and deployment phases of the web application. [6, 7, 8, 9, 10]

1.2 Project Scope

The platform is a fully responsive web application enabling businesses to list products and customers to browse, cart, and securely purchase goods online. It includes an integrated content management system for inventory, order fulfillment, and basic data analytics. [11, 12, 13, 14, 15]

1.3 Intended Audience

- **Development Team:** To engineer frontend components, backend APIs, and database schemas.
- **QA Team:** To design unit tests, integration tests, and user acceptance criteria.
- **Project Managers:** To track milestone progress and prevent scope creep. [16]

2. Overall Description

2.1 Product Perspective

The system operates as a distributed multi-tier web architecture. It communicates with external third-party services via REST APIs for credit card processing, shipping calculation, and transactional emails. [17]

2.2 User Classes and Characteristics

- **Guest Users:** Unauthenticated visitors who browse catalogs, read reviews, and add items to a temporary cart.
- **Registered Customers:** Authenticated users with access to saved shipping addresses, order histories, and wishlists.

- **Store Administrators:** Privileged staff who manage product inventory, view revenue metrics, and update order statuses. [18, 19, 20, 21]
-

3. Functional Requirements

3.1 User Authentication and Profiles

- **Registration:** Users must register using an email, Google single sign-on, or Apple ID.
- **Password Policies:** Passwords must meet a minimum length of 8 characters and include at least one number and special symbol.
- **Session Management:** Users must remain logged in for up to 30 days unless they explicitly sign out or clear cookies. [22]

3.2 Product Catalog and Search

- **Dynamic Filtering:** Customers must be able to filter search results by price range, brand, average rating, and availability.
- **Elastic Search:** The system must provide autocomplete suggestions within 200 milliseconds of user input.
- **Media Management:** Product pages must render up to 5 high-resolution images and one optional video per SKU. [23, 24]

3.3 Shopping Cart and Checkout

- **Persistent Carts:** The shopping cart must retain unpurchased items for registered users across multiple device sessions.
 - **Tax Calculation:** The checkout module must compute localized sales tax dynamically based on the destination zip code.
 - **Payment Gateways:** The platform must securely process payments through Stripe and PayPal. [25]
-

4. Non-Functional Requirements

4.1 Security Requirements

- **Data Encryption:** All transit data must utilize Transport Layer Security (TLS 1.3), and database passwords must use bcrypt hashing.
- **PCI Compliance:** The system must never store raw credit card numbers or CVV codes on local databases to maintain Payment Card Industry Data Security Standard compliance. [26]

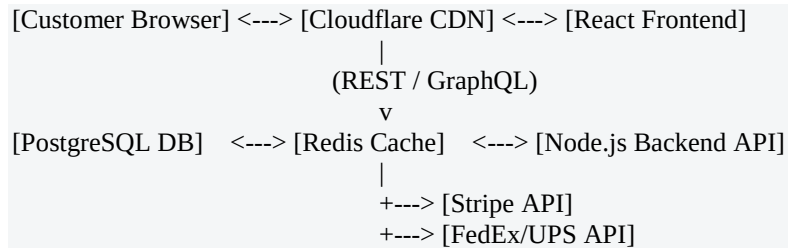
4.2 Performance and Scalability

- **Page Load:** The homepage and product detail pages must achieve a Google Lighthouse performance score above 85, loading the largest contentful paint within 2.5 seconds.
- **Concurrency:** The infrastructure must support up to 10,000 concurrent active user sessions without degradation in server response time. [27, 28, 29]

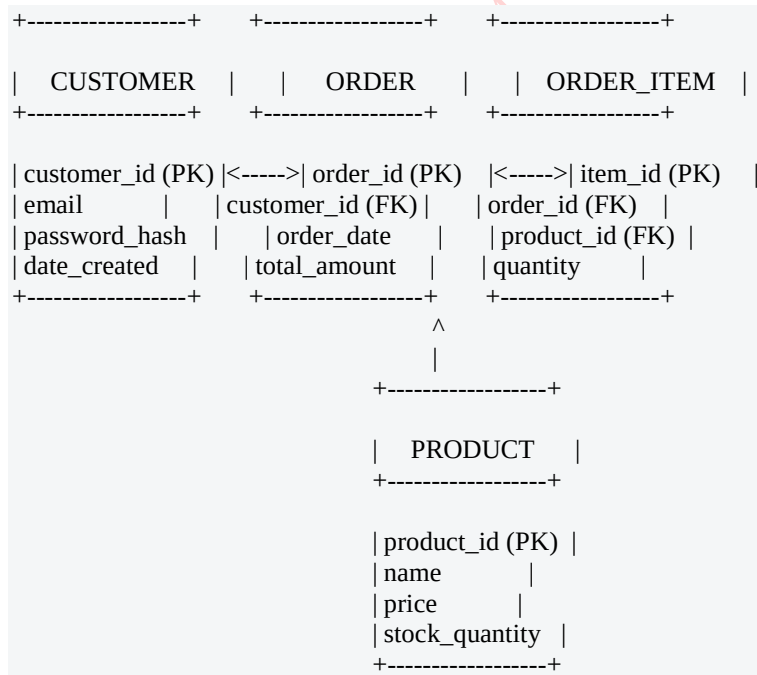
4.3 Availability

- **Uptime Objective:** The web application must maintain a 99.9% uptime availability metric, excluding pre-announced monthly maintenance windows.

5. System Architecture & Data Flow



5.1 Entity Relationship Diagram Core



2. PROJECT DESCRIPTION:

Here is a comprehensive **Project Description** tailored for an E-Commerce Website. This document provides a high-level, business-oriented overview suitable for stakeholders, investors, and project kick-offs.

2. Project Description: Next-Generation E-Commerce Platform

2.1 Problem Statement

Modern consumers expect seamless, lightning-fast online shopping experiences, yet many small-to-medium retail brands still rely on rigid legacy platforms. These outdated systems suffer from slow mobile loading times, poor search capabilities, and fragmented checkout processes, which collectively drive up shopping cart abandonment rates and limit business growth.

2.2 Project Vision & Goal

The goal of this project is to develop a high-performance, fully responsive **B2C E-Commerce Platform** that bridges the gap between retail brands and digital consumers. The platform will deliver an intuitive, mobile-first marketplace optimized for speed, discoverability, and frictionless purchasing, ultimately maximizing customer conversion rates and brand loyalty.

2.3 Key Deliverables

- **Customer Web Application:** A responsive frontend website featuring an optimized storefront, interactive product catalog, persistent shopping cart, and unified checkout.
- **Merchant Admin Dashboard:** A secure portal enabling store administrators to manage inventory, fulfill customer orders, update pricing, and generate sales reports.
- **Backend API Infrastructure:** A scalable API engine linking frontend actions to database systems, payment gateways, and third-party logistics.

2.4 Target Audience

- **Primary Consumers:** Tech-savvy retail shoppers seeking an efficient, secure, and personalized online shopping experience across mobile, tablet, and desktop devices.
- **Store Operations Team:** Internal managers, inventory specialists, and customer support representatives who require streamlined tools to manage daily business workflows.

2.5 Strategic Business Objectives

- **Optimize Performance:** Achieve sub-2-second page load speeds globally to minimize user drop-off.

- **Boost Checkout Conversion:** Reduce overall shopping cart abandonment by 15% through a simplified, secure 3-step checkout sequence.
- **Ensure Data Security:** Implement industry-standard protocols to guarantee 100% secure payment processing and safe storage of consumer user profiles.

FINAL YEAR PROJECT BY M WASEEM AKHTAR & M ALI DANISH BSCS

3. PROJECT WORKLOAD:

This Project Workload framework details the estimated human effort, core tasks, and resource allocation required to design, develop, and deploy the E-Commerce Website. It translates the project scope into operational hours based on an agile development lifecycle.

1. Workload Estimation Summary

The total estimated workload for a standard Minimum Viable Product (MVP) e-commerce website is **720 developer/designer hours**. This estimation assumes a cross-functional team working concurrently across a typical 12-week timeline.

Project Phase	Estimated Effort	Primary Resource Role
UI/UX Design	120 Hours	Product Designer
Frontend Development	200 Hours	Frontend Engineer
Backend & DB Engineering	240 Hours	Backend Engineer
Third-Party Integrations	80 Hours	Integration Specialist
QA Testing & Deployment	80 Hours	QA Engineer

2. Work Breakdown Structure (WBS) & Task Allocation

2.1 UI/UX Design Phase (120 Hours)

- **User Journeys:** Mapping customer paths from landing page to checkout (**20 Hours**).
- **Wireframing:** Creating low-fidelity skeletal layouts of the catalog and dashboard (**40 Hours**).
- **High-Fidelity Mockups:** Designing final visual interfaces, typography, and dark/light modes (**60 Hours**). [2, 3, 4, 5]

2.2 Frontend Development Phase (200 Hours)

- **Storefront Componentry:** Coding reusable product cards, grids, and navigation bars (**60 Hours**).
- **State Management:** Configuring global app states for persistent shopping carts (**50 Hours**).
- **Responsive Layouts:** Optimizing views for mobile, tablet, and desktop viewports (**50 Hours**).

- **Admin UI Layout:** Building tables, sidebar navigation, and analytics charts for store managers (40 Hours).

2.3 Backend & Database Engineering Phase (240 Hours)

- **Schema Architecture:** Creating relational database models for users, orders, and products (40 Hours).
- **Auth Infrastructure:** Coding JWT generation, secure hashing, and OAuth connections (40 Hours).
- **Search Engine Logic:** Implementing indexed backend search, filtering, and sort queries (60 Hours).
- **Order Business Logic:** Engineering stock deduct mechanics, invoice builders, and cart math (60 Hours).
- **Admin API Controllers:** Programming endpoints for product creation, modification, and sales exports (40 Hours).

2.4 Integration Phase (80 Hours)

- **Payment Gateways:** Connecting Stripe webhooks and processing pipeline validations (35 Hours).
- **Shipping APIs:** Integrating FedEx/UPS live rate calculation engine mechanisms (25 Hours).
- **Communications:** Setting up transactional mail/SMS nodes via Twilio or SendGrid (20 Hours). [6]

2.5 QA Testing & DevOps Phase (80 Hours)

- **Unit & Integration Tests:** Writing backend controller checks and schema assertions (30 Hours).
- **E2E Testing:** Running automated user scripts simulating full cart checkout procedures (20 Hours).
- **CI/CD Pipeline Build:** Configuring cloud deployment containers and CDN caching networks (30 Hours).

3. Resource Allocation & Capacity Matrix

To complete this 720-hour workload within a **12-week schedule**, tasks are distributed among a 4-person production squad. Each team member contributes roughly 15 to 20 hours of focused production work per week toward this specific project:

[12-Week Production Timeline]

Weeks 1-3 : 🎨 UI/UX Designer =====> (120 Hours Total)

Weeks 3-9 : 💻 Frontend Dev =====> (200 Hours Total)

Weeks 3-10 : ⚙ Backend Dev =====> (240 Hours Total)

Weeks 9-11 : 🛠 Backend Dev (Int) =====> (80 Hours Total)

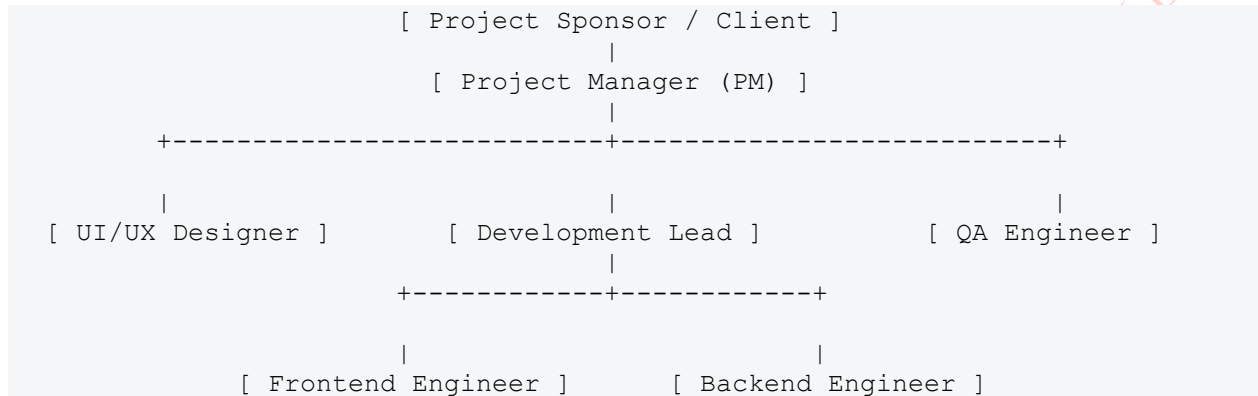
Weeks 10-12 : 🛡 QA & DevOps =====> (80 Hours Total)

- **Product Designer:** Allocates **40 hours/week** during Weeks 1–3 to lock in interfaces before coding starts.
- **Frontend Engineer:** Allocates **33 hours/week** during Weeks 3–9 to build out the operational interface components.
- **Backend Engineer:** Allocates **30 hours/week** across Weeks 3–11 handling databases, core logic, and third-party APIs.
- **QA/DevOps Engineer:** Allocates **40 hours/week** during Weeks 10–12 ensuring everything functions perfectly under stress before launch.

4. PROJECT MEMBERS:

This **Project Members** framework defines the roles, responsibilities, and resource requirements for the E-Commerce Website project. It outlines who is responsible for executing the 720-hour workload detailed in the previous section.

1. Project Organization Chart



2. Core Project Roles and Responsibilities

An e-commerce launch requires a balanced, cross-functional team. Below are the key project members and their specific ownership areas:

2.1 Project Manager (PM)

- **Responsibilities:** Acts as the central point of contact between stakeholders and the production team. Tracks sprint milestones, handles risk mitigation, and manages budget and timeline tracking.
- **Key Deliverable:** Weekly status reports, Gantt charts, and prioritized Jira boards.

2.2 UI/UX Product Designer

- **Responsibilities:** Conducts competitor research and maps out customer shopping journeys. Designs high-fidelity visual screens for both responsive web viewports (mobile, tablet, desktop) and the store admin dashboard.
- **Key Deliverable:** Figma design system, interactive prototypes, and asset export packages.

2.3 Lead Frontend Engineer

- **Responsibilities:** Translates static designs into interactive web experiences. Implements responsive layouts, manages global state for persistent shopping carts, and optimizes web assets to ensure fast page speeds.
- **Key Deliverable:** Clean HTML/CSS/JavaScript components, client-side API integrations, and public storefront pages.

2.4 Lead Backend Engineer & DevOps

- **Responsibilities:** Architects relational database schemas for users, products, and orders. Engineers secure APIs, implements system user authentication (JWT), configures third-party pipelines (Stripe, shipping calculators), and establishes cloud hosting architecture.
- **Key Deliverable:** REST or GraphQL APIs, encrypted database infrastructure, and functional CI/CD pipeline automation.

2.5 Quality Assurance (QA) Engineer

- **Responsibilities:** Writes test cases to catch visual and functional bugs across multiple web browsers. Executes end-to-end testing scripts on the payment checkout gateway, and validates non-functional security and platform traffic requirements.
- **Key Deliverable:** Automated test suites, bug tracking logs, and a final compliance verification sign-off sheet.

3. Resource Responsibility Assignment Matrix (RACI)

This matrix maps out how project members interact with major project deliverables:

- **R (Responsible):** The person who physically executes the task.
- **A (Accountable):** The person with final decision-making power and ownership over success.
- **C (Consulted):** Subject matter experts whose input is gathered prior to work.
- **I (Informed):** Stakeholders kept up to date on progress but not actively involved.

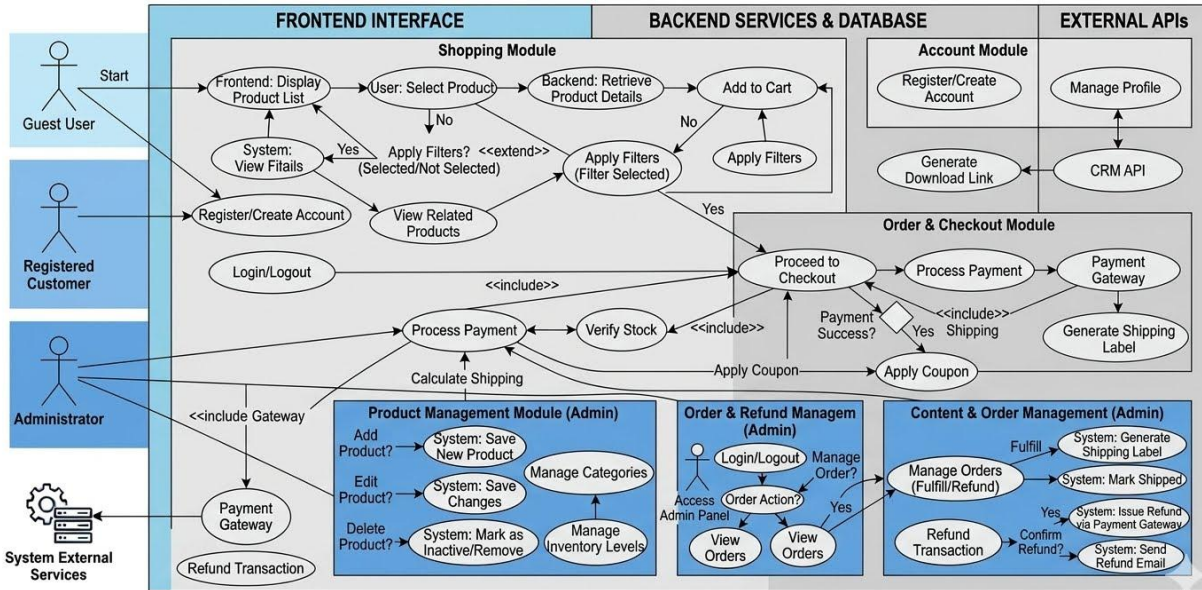
Deliverable / Task	Project Manager	UI/UX Designer	Frontend Eng.	Backend Eng.	QA Engineer
SRS Document Sign-Off	A	C	C	C	C
UI/UX Wireframes	I	R / A	C	I	I

Database Schema Design	I	I	C	R / A	I
Payment Gateway Checkout	I	C	R	R / A	I
End-to-End System Testing	I	I	C	C	R / A
Production Launch	A	I	R	R	R

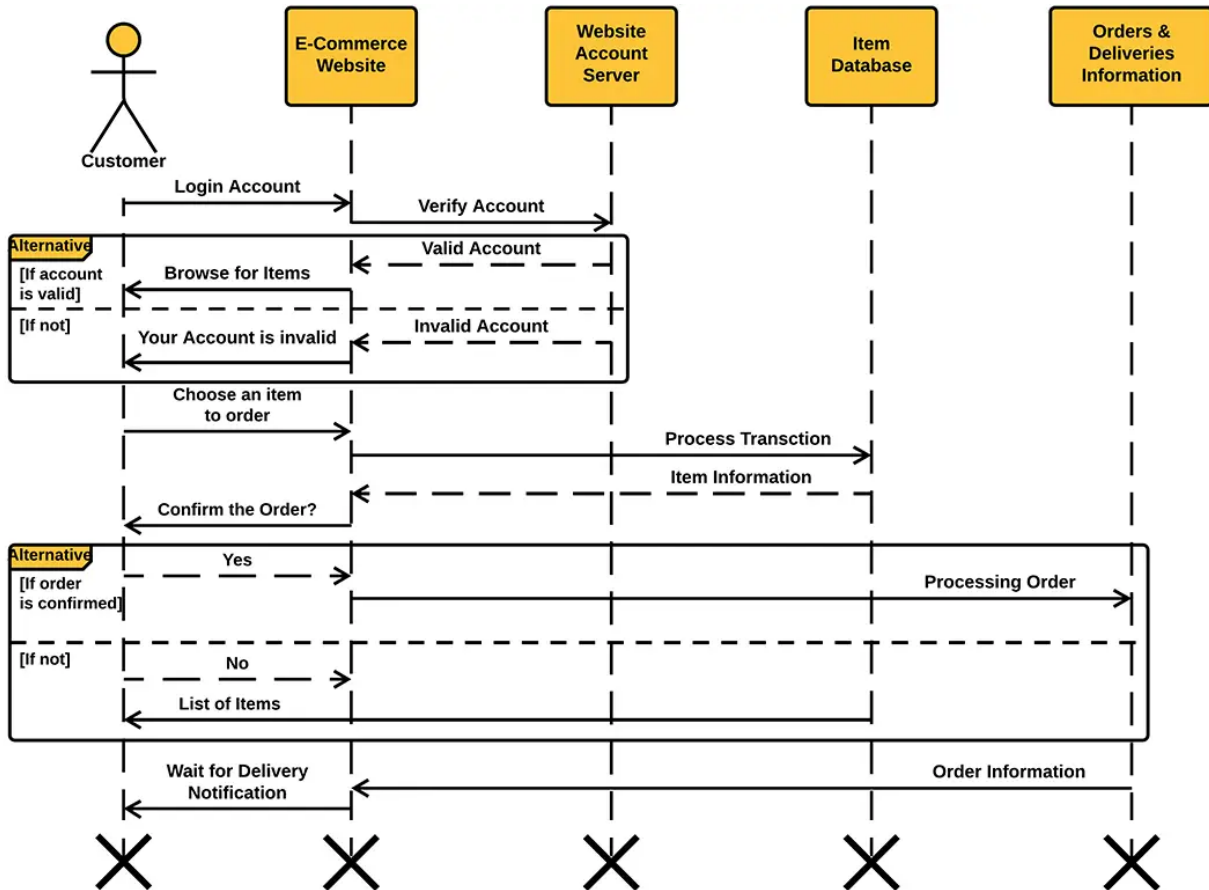
FINAL YEAR PROJECT BY M WASEEM AKHTAR & M ALI DAMISH BSCS

6. ACTIVITY DIAGRAM:

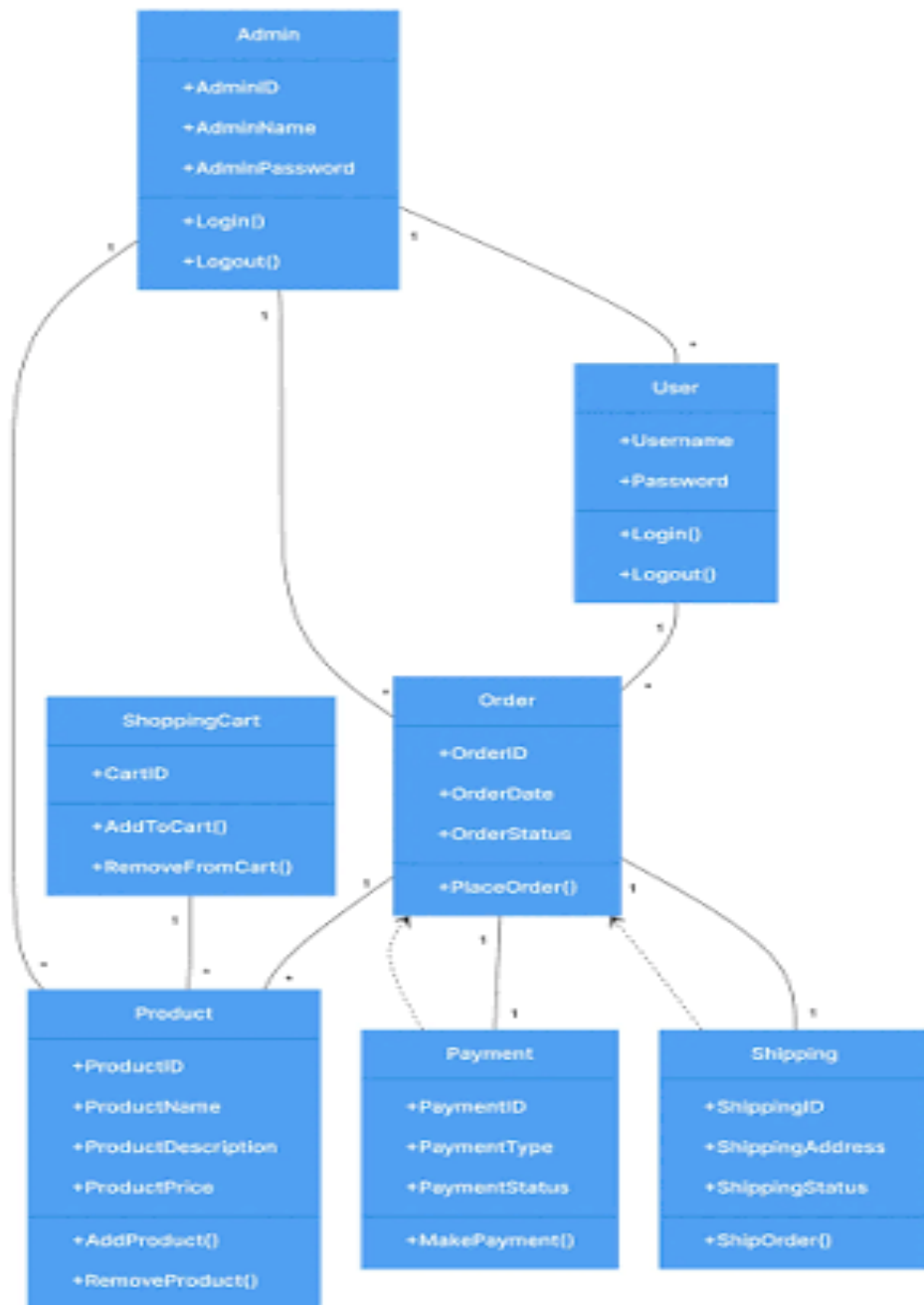
E-Commerce Website System Activity Diagram



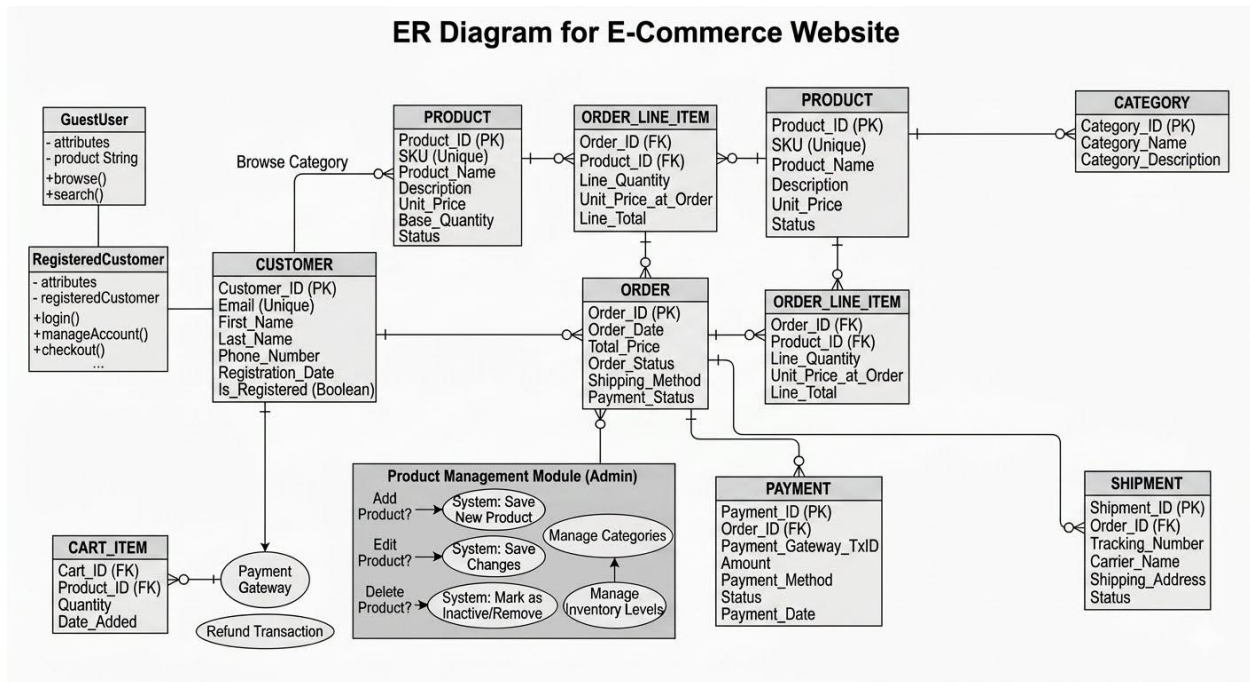
7. ACTIVITY DIAGRAM:



8. CLASS DIAGRAM:



9. ER DIAGRAM:

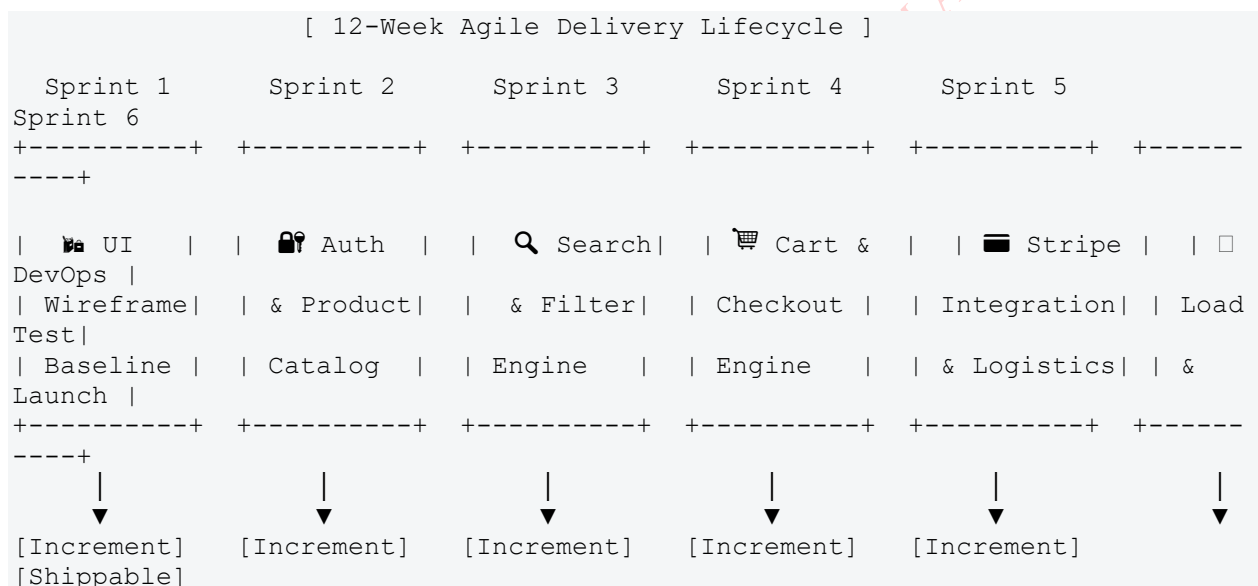


10. PROCESS MODEL, SCRUM,/AGILE:

This **Agile Scrum Process Model** details how the E-Commerce Website project will be executed dynamically across interactive iterations. It outlines the specific artifacts, sprint rhythms, and user stories required to build and deploy the platform incrementally.

1. The E-Commerce Sprint Strategy

The development workload is broken down into **six consecutive 2-week sprints** using an incremental process model. Rather than launching all components at once, each sprint delivers a testable, production-ready slice of the platform.



2. Core Scrum Ceremonies for the Project

To maintain development velocity, the cross-functional team adheres to a strict sprint battle rhythm:

- **Sprint Planning (First Monday of Sprint, 2 hours):** The Product Owner defines high-priority items from the product backlog. The engineers break down these items into tasks and commit to a specific sprint goal (e.g., *"Enable secure credit card processing via Stripe"*).
- **Daily Scrum (Mon–Fri, 15 minutes):** A timeboxed sync where developers answer three core questions:
 1. What did I complete yesterday to help meet the sprint goal?

2. What will I work on today?
 3. Are there any technical blockers in my way?
- **Sprint Review (Last Friday of Sprint, 1 hour):** The team demonstrates a live, working increment of the e-commerce site to stakeholders (e.g., demonstrating a functional user registration or cart page).
 - **Sprint Retrospective (Last Friday of Sprint, 1 hour):** The Scrum Master facilitates a review session to determine what went well, what went wrong, and actions to improve workflow efficiency for the next sprint.
-

3. Product Backlog & Sample User Stories

The Product Owner manages the e-commerce features as discrete, customer-centric **User Stories**. Each story follows standard Agile syntax and must pass rigorous acceptance criteria to be marked as complete.

User Story 1: Persistent Shopping Cart (Sprint 4 Priority)

- **As a** registered customer,
- **I want to** add items to my shopping cart and close the browser,
- **So that** my items are still saved when I log back in later on another device.
- **Acceptance Criteria:**
 - Cart items must link to the user's database ID when authenticated.
 - Unauthenticated guest carts must persist via local storage tokens for 7 days.
 - The cart badge must update dynamically without forcing a full page refresh.

User Story 2: Secure Payment Processing (Sprint 5 Priority)

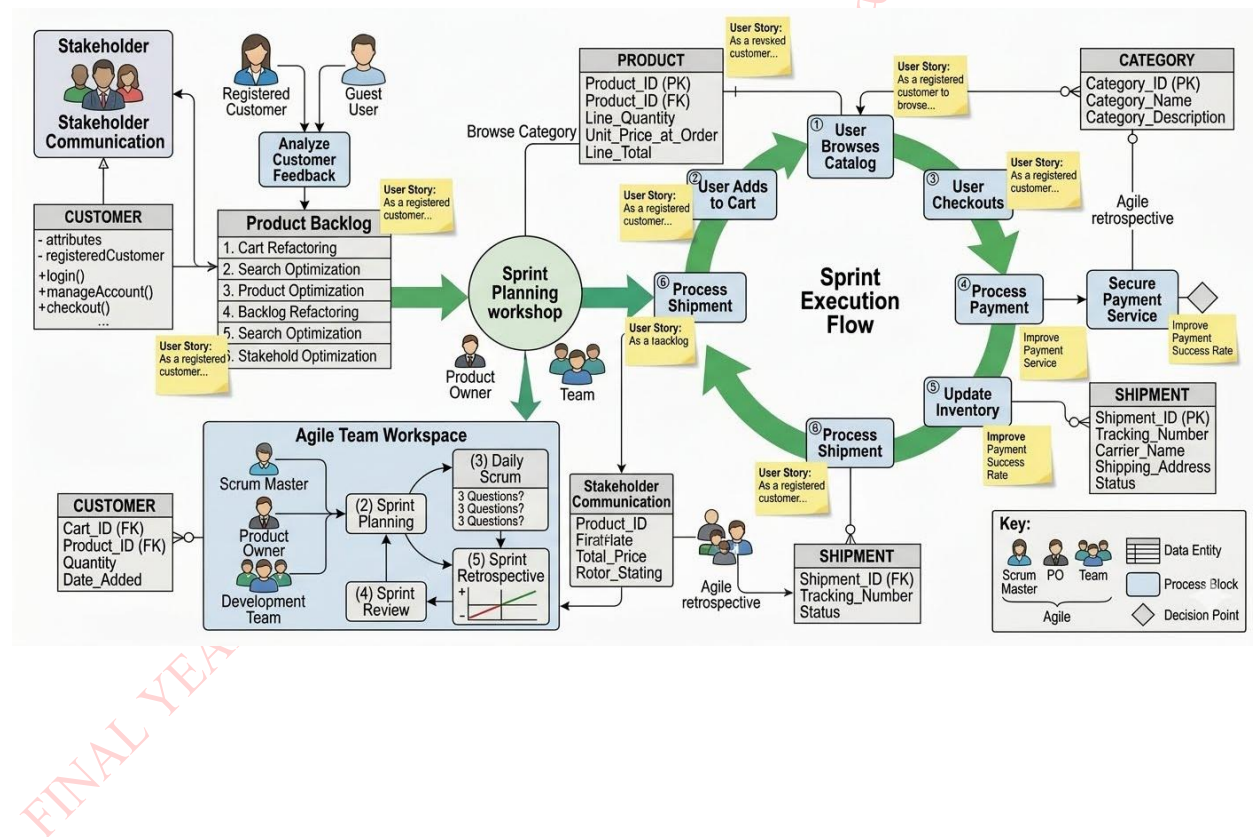
- **As a** buyer check out on the web storefront,
 - **I want to** input my credit card credentials securely,
 - **So that** I can confidently complete my financial transaction.
 - **Acceptance Criteria:**
 - Credit card forms must use Stripe Elements to avoid handling raw financial data.
 - The system must handle payment failure errors gracefully and display helpful alerts.
 - On payment success, an entry must generate in the `ORDER` table, and a confirmation email must trigger.
-

4. Definition of Done (DoD)

Before any e-commerce user story can transition from "In Progress" to "Done" at the end of a sprint, it must pass the following standardized quality gate:

- **Code Quality:** Peer reviewed by at least one other engineer with zero critical linter warnings.
- **Testing:** Unit test coverage meets or exceeds 80% for backend route modifications.
- **Security:** No hardcoded API keys; checkout processes conform to strict PCI compliance guidelines.
- **Accessibility:** Storefront interface components conform to WCAG 2.1 AA design contrast compliance.

4. Process Model and Agile/Scrum for E-Commerce Platform Diagram:



11. GANTT CHART:

This **Gantt Chart Schedule** details the production timeline, sequential task paths, and deployment milestones for the E-Commerce Website across a **12-week development lifecycle**.

1. 12-Week Visual Project Schedule

=====											
=====											
PHASE / TASK NAME	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	
W11 W12 DEPENDENCY											
=====											
=====											
PHASE 1: RESEARCH & UI											
1.1 SRS Document Lock		[==]									
		None									
1.2 Wireframes & Mockups			[=====]								
		1.1									
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----											
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----											
PHASE 2: FOUNDATION											
2.1 DB Schema & Models				[==]							
		1.1									
2.2 Auth & JWT Logic					[==]						
		2.1									
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----											
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----											
PHASE 3: CORE FE & BE											
3.1 Storefront Screens				[=====]							
		1.2									
3.2 Search & Filters					[=====]						
		2.1, 3.1									
3.3 Cart & Order Engine						[=====]					
		3.2									
3.4 Admin Panel UI							[=====]				
		1.2									
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----											
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----											
PHASE 4: INTEGRATIONS											
4.1 Stripe Gateway Node								[=====]			
		3.3									
4.2 Shipping & Mail APIs									[==]		
		3.3									
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----											
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----											
PHASE 5: QA & DEPLOY											

5.1 End-to-End Testing												[===]
		4.1, 4.2										
5.2 Cloud Deploy & CDN												
[===]		5.1										
5.3 Production Launch												
	🔗	5.2										
=====												
=====												

2. Key Project Milestones

Milestones represent critical check-gates that require verification before entering subsequent phases:

- **Milestone 1 (End of Week 2): Design Freeze.** Figma mockups are approved; frontend developers can safely code interfaces without facing design modifications.
- **Milestone 2 (End of Week 4): Backend Foundation Ready.** Relational schemas and authentication systems are live and verified.
- **Milestone 3 (End of Week 9): Feature Complete.** Core e-commerce mechanics (search, cart, check out, and integrations) are fully functional on local staging environments.
- **Milestone 4 (End of Week 12): Production Launch.** The platform passes security compliance filters and goes live to the public.

3. Managing Task Dependencies

To protect the launch timeline, project managers monitor the **critical path** created by task linkages:

- **Design to Code Integration:** The **Storefront Screens (3.1)** depend entirely on **Wireframes & Mockups (1.2)**. If design deliverables slip by 3 days, the frontend code start date slips uniformly.
- **Transactional Architecture Integration:** The **Stripe Gateway Node (4.1)** cannot collect payment values until the backend calculation logic in the **Cart & Order Engine (3.3)** is finalized.
- **Staging Validation Gate: End-to-End Testing (5.1)** cannot commence while external transactional structures are unlinked, as automated QA routines simulate a complete customer journey from search to final invoicing.

OBLIGED

FINAL YEAR PROJECT BY M WASEEM AKHTAR & M ALI DANISH BSCS